

Beyond Autocomplete: A Survey on the Evolution of Human-AI Pair Programming

Nikhil Anand

University of Massachusetts, Amherst

nikhilanand@umass.edu

Abstract

Generative AI tools such as GitHub Copilot and ChatGPT are increasingly integrated into software development workflows, yet their influence on developers' problem solving strategies remains insufficiently understood. This paper presents a literature survey of ten recent studies examining how generative AI affects developer behavior during programming tasks. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted interactions and then analyzes their impact across key stages of problem-solving, including problem understanding, solution generation, implementation, and debugging. Our synthesis shows that generative AI shifts developers from exploration-driven problem-solving toward solution selection and refinement, reducing cognitive effort while introducing risks of over-reliance and shallow reasoning. The impact varies across stages: conversational tools primarily support problem understanding, while suggestion-based and generative tools accelerate implementation and debugging. We further identify cross-cutting patterns in trust, prompting behavior, and learning. Finally, we highlight gaps in existing work, including limited longitudinal evidence and insufficient analysis of expertise-dependent effects. These findings provide a structured perspective on how generative AI reshapes developer problem-solving and suggest directions for future research.

CCS Concepts • **Software and its engineering** → **Software development process management**; • **Computing methodologies** → *Multi-agent systems*; Information extraction; Natural language generation

1. Introduction

Generative AI tools powered by large language models (LLMs), such as GitHub Copilot and ChatGPT, are

rapidly becoming integral to software development workflows. These tools provide a wide range of capabilities, including code completion, natural language driven code generation, explanation of programming concepts, and automated debugging assistance. Unlike traditional code recommendation systems, Generative AI enables interactive and iterative engagement through conversational interfaces, allowing developers to specify intent, refine outputs, and explore alternative solutions in real time [1, 14]. Recent empirical studies report widespread adoption of such tools and demonstrate measurable improvements in task completion speed and perceived productivity [2, 11]. As a result, Generative AI is reshaping how developers interact with code and development environments.

Beyond productivity improvements, emerging research has begun to characterize how Generative AI influences developer behavior during programming tasks. Studies have observed changes in code writing practices, debugging strategies, and patterns of interaction with development tools [1, 7]. Developers increasingly engage in prompt based workflows, iteratively refining natural language queries to obtain desired outputs, and often integrate AI-generated code directly into their programs with varying levels of verification [6, 14]. These interactions introduce new dynamics in programming, including shifts in how solutions are explored, how alternatives are evaluated, and how correctness is established. At the same time, prior work highlights variability in usage patterns across developers, tasks, and tool modalities, reflecting differences in expertise, trust, and familiarity with AI-assisted programming environments [6, 7].

To provide a structured understanding of these developments, this paper presents a literature survey of ten recent studies examining developer interactions with Generative AI tools. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted programming interactions, capturing distinct modes of engagement such as suggestion-based, conversational, and generative workflows. Building on this taxonomy, we analyze how these interaction types influence different stages of the software development problem-solving process, including problem understanding, solution generation, implementation, and debug-

ging. By synthesizing findings across diverse empirical settings and study designs, this work offers a consolidated view of how Generative AI tools shape developer behavior and interaction patterns in programming tasks.

1.1 Motivation

The dream of an "AI assistant" working alongside the programmer has captured our imagination for several decades now, giving rise to a rich body of work from both the programming languages [5, 9, 13] and the machine learning [Guo et al. 2021; Kalyan et al. 2018; Xu et al. 2020] communities. Thanks to recent breakthroughs in large language models (LLMs) [Li et al. 2022; Vaswani et al. 2017] this dream finally seems within reach. OpenAI's Codex model [Chen et al. 2021], which contains 12 billion model parameters and is trained on 54 million software repositories on GitHub, is able to correctly solve 30–70% of novel Python problems, while DeepMind's AlphaCode [Li et al. 2022] ranked in the top 54.3% among 5000 human programmers on the competitive programming platform Codeforces. With this impressive performance, large code generating models are quickly escaping research labs to power industrial programming assistant tools, such as Github Copilot [Friedman 2021].

Generative AI has emerged as a transformative paradigm across a wide range of domains, enabling systems to produce human-like text, code, and other artifacts with increasing fidelity. Advances in large language models (LLMs) have demonstrated strong capabilities in reasoning, language understanding, and generation, leading to their rapid adoption in real-world applications [3, 4]. These models have shifted the role of AI from purely analytical systems to interactive assistants capable of supporting complex cognitive tasks. As a result, Generative AI is increasingly positioned as a collaborative partner in knowledge work, fundamentally altering how users interact with computational systems and solve problems.

In the context of software development, Generative AI tools such as GitHub Copilot and ChatGPT provide direct support for programming activities, including code generation, explanation, and debugging. Empirical studies have shown that these tools can improve developer productivity, reduce task completion time, and assist in learning unfamiliar concepts [6, 11, 14]. Developers are able to offload routine implementation effort and leverage AI-generated suggestions to accelerate development workflows. At the same time, these tools introduce new interaction patterns, such as prompt-based programming and iterative refinement, which reshape how developers approach coding tasks. This evolution raises important questions about how developers engage with problems, evaluate solutions, and integrate AI assistance into their workflows.

At an industry level, the integration of Generative AI into development environments has significant implications for productivity, collaboration, and software engineering prac-

tices. Organizations are increasingly adopting AI-assisted programming tools to improve efficiency and reduce development costs, while also redefining skill requirements for developers [2, 8]. Despite these advancements, there remains limited understanding of how Generative AI influences the underlying problem-solving strategies of developers across different stages of programming. Existing studies often focus on performance metrics or usability, leaving a gap in understanding the cognitive and behavioral shifts induced by AI assistance. This highlights the need for a structured synthesis of current research to better understand how Generative AI reshapes developer problem-solving and to inform the design of future tools and practices.

2. AI-Assisted Programming

To systematically analyze how generative AI influences developer problem-solving, we first characterize the different modes through which developers interact with AI systems during programming tasks. Rather than categorizing tools based on underlying models or capabilities, we focus on interaction patterns that reflect how developers engage with AI assistance in practice. In particular, we distinguish between two primary modes of interaction: exploration and acceleration. Exploration-oriented interactions involve using AI to understand problems, generate ideas, and examine alternative solutions, often through iterative and conversational engagement. In contrast, acceleration-oriented interactions focus on efficiently completing tasks by leveraging AI-generated suggestions or code with minimal deliberation. This distinction provides a useful lens for analyzing how AI support shapes developer behavior across different stages of problem-solving.

Generative AI as an Autocomplete Tool When solving structured programming tasks such as those found in Advent of Code [15], developers often decompose the problem into smaller subtasks, such as parsing input into appropriate data structures. Even when the solution approach is clear, generative AI tools can be used to accelerate implementation by assisting with code completion and reducing the effort of recalling syntax or API details. Developers may provide contextual comments to guide the tool, after which inline suggestions appear during pauses in typing. These suggestions, including end-of-line completions (Figure 1), enable faster code writing while maintaining the developer's original intent. In this case, Copilot suggests the correct API function invocation to split the rule into its left- and right-hand sides. To come up with this suggestion, Copilot relies on the context, i.e. some amount of source file content preceding the cursor, which can include both code and natural language comments, as is the case in our example.

Generative AI as an Exploration Tool When developers encounter unfamiliar tasks, generative AI can be used as an exploration tool to understand both the problem and potential solution approaches. Developers may begin by express-

```

# rules are formatted like:
# AB => C
def parse_input(filename):
    with open(filename) as f:
        template, rules = f.read().split("\n\n")
        for rule in rules:
            rule_parts = rule.split(" => ")

```

Figure 1: An example of an end-of-line suggestion from Copilot, which completes the current line of code based on the context provided by preceding code and comments.

```

You, now | 1 author (You)
1  ✓ import matplotlib
2  import matplotlib.pyplot as plt
3
4  ✓ def read_first_digits_from_file(filename):
5  ✓     with open(filename) as file:
6      data = file.read().splitlines()
7      return [int(line[0]) for line in data]
8
9  fib_first_digits = read_first_digits_from_file("fib.")
10 inverse_first_digits = read_first_digits_from_file("fib.")
11
12 # Plot the first digits of the Fibonacci
13 # sequence as a histogram
14
15
16
17

```

Figure 2: Example code where a user needs a more lengthier suggestion

ing the task through natural language prompts (lines 12-13 of Figure 2), which enables the system to generate multiple candidate solutions. By invoking the multi-suggestion interface (Figure 3), they can examine several alternatives, identify common patterns, and observe relevant API usage, helping build confidence in the generated outputs. This process allows developers to select suitable code snippets while also gaining an understanding of underlying abstractions and libraries. To ensure correctness, the generated code is typically executed and inspected, serving as a form of validation. Such interactions are iterative and deliberate, involving prompt refinement, comparison of alternatives, and verification, reflecting an exploratory mode of problem-solving.

3. Taxonomy of AI-Assisted Programming

3.1 Suggestion-Based Interactions

Suggestion-based assistance represents a class of AI interactions where developers receive real-time code completions based on the current programming context. These interactions are typically lightweight and minimally intrusive, requiring little to no explicit input beyond the code being written. By providing inline or on-demand suggestions, such systems aim to accelerate implementation by reducing

```

-
3  =====
4
5  Accept Solution
6  # Plot the first digits of the Fibonacci sequence as
7  plt.hist(fib_first_digits, bins=range(0, 10))
8  plt.title("Fibonacci sequence")
9  plt.xlabel("First digit")
10 plt.ylabel("Number of occurrences")
11 plt.savefig("fib.png")
12
13 =====
14
15 Accept Solution
16 # Plot the first digits of the Fibonacci sequence as
17 plt.hist(fib_first_digits, bins=range(0, 10))
18 plt.title("Fibonacci sequence")
19 plt.xlabel("First digit")
20 plt.ylabel("Number of occurrences")
21 plt.show()
22
23 =====
24
25 Accept Solution
26 # Plot the first digits of the Fibonacci sequence as
27 plt.hist(fib_first_digits, bins=10, range=(0, 10))
28 plt.title("Fibonacci sequence")
29 plt.xlabel("First digit")
30 plt.ylabel("Number of occurrences")
31 plt.savefig("fib.png")
32

```

Figure 3: An example of Generative AI being used as an exploration tool, where the developer provides a natural language prompt to generate multiple candidate solutions.

keystrokes, aiding recall of syntax and APIs, and streamlining routine coding tasks. In this section, we categorize suggestion-based assistance into distinct subtypes based on the scope and presentation of generated code.

3.1.1 End-of-Line Completion

End-of-line completion represents the most granular form of suggestion-based assistance, where the AI system predicts and completes the remainder of a partially written line of code. These suggestions are typically short, context-aware, and require minimal cognitive overhead to evaluate, enabling developers to maintain flow during implementation. As illustrated in Figure 1, such completions often rely on local context and common coding patterns to provide syntactically correct continuations. Prior work shows that developers frequently accept these suggestions due to their low cost of verification and seamless integration into the coding process [1]. This interaction mode primarily reduces keystrokes and supports rapid code construction without significantly altering higher-level problem-solving strategies.

3.1.2 Multi-Line / Block Completion

Multi-line or block completion extends suggestion-based assistance by generating larger code segments, such as entire functions, loops, or conditional structures, based on the

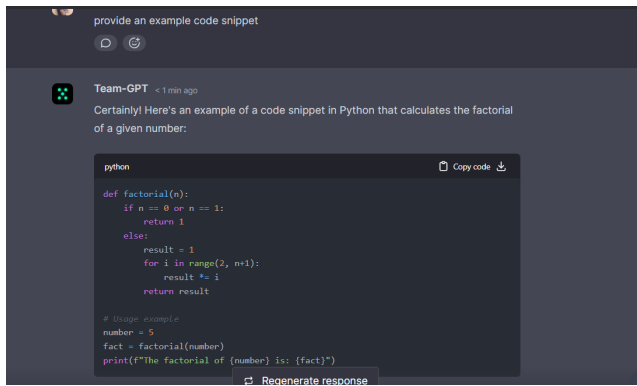


Figure 4: An example of conversational assistance, where the developer engages in a natural language dialogue with the AI to explore the problem and refine solutions.

surrounding context and developer intent. These suggestions provide more substantial acceleration benefits by automating repetitive or boilerplate code generation, allowing developers to focus on higher-level logic. However, larger completions require greater cognitive effort to evaluate and verify, as they may introduce subtle errors or assumptions [6, 14]. Studies indicate that while developers benefit from increased productivity, they may also rely on these suggestions without fully understanding the generated code, highlighting a trade-off between efficiency and comprehension [6].

3.1.3 Context-Aware Pattern Completion

Context-aware pattern completion refers to suggestions that capture recurring coding patterns or idioms across files or projects, enabling the AI system to generate code that aligns with broader structural or stylistic conventions. These suggestions go beyond local syntax completion by incorporating higher-level contextual signals, such as variable usage, naming conventions, or previously implemented logic. As a result, they can assist in maintaining consistency and reducing redundancy in codebases [13]. However, reliance on such patterns may reinforce existing coding habits and limit exploration of alternative implementations, subtly influencing developer decision-making during implementation.

3.2 Conversational Assistance

Conversational assistance refers to interaction modes in which developers engage with generative AI through natural language dialogue (Figure 4) to explore problems, request explanations, or refine solutions [6, 14]. Unlike suggestion-based systems, these interactions are explicit and iterative, allowing developers to externalize their reasoning and guide the AI through prompts and follow-up queries. Prior work shows that such conversational workflows enable flexible problem exploration and support tasks such as understanding unfamiliar code, learning APIs, and clarifying requirements.

Conversational assistance can be broadly categorized into query-driven interaction and iterative refinement.

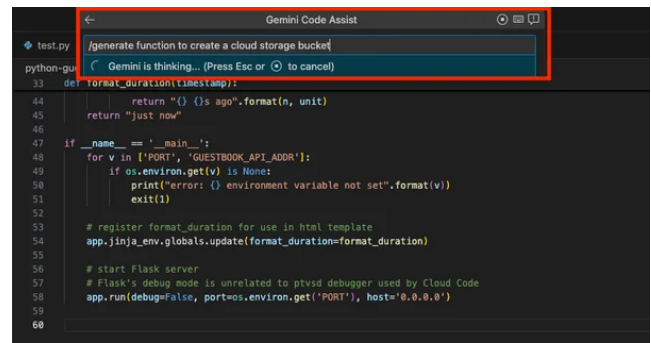


Figure 5: An example of generative assistance, where the developer provides a high-level intent and receives a substantial code output that implements the desired functionality.

1. In **query-driven interaction**, developers pose direct questions (e.g., requesting explanations or examples), using the AI as an on demand knowledge source.
2. In **iterative refinement**, developers progressively adjust prompts based on previous responses, engaging in a back-and-forth dialogue to converge on a desired solution.

These subcategories support exploration-oriented workflows by enabling developers to decompose problems, compare alternatives, and refine their understanding, although the quality of outcomes depends heavily on prompt formulation and interpretation [1].

3.3 Generative / Transformational Assistance

Generative or transformational assistance refers to interaction modes where developers provide high-level intent and receive substantial code outputs, such as functions, program fragments, or refactored implementations. These interactions abstract away low-level details, allowing developers to focus on specifying desired functionality while the AI produces corresponding code artifacts (Figure 5). Such capabilities are central to modern LLM-based tools and are widely used to accelerate implementation and reduce manual coding effort [6, 11].

This form of assistance can be divided into code generation and code transformation. Code generation involves producing new code from natural language descriptions or partial context, while code transformation focuses on modifying existing code, such as refactoring, optimizing, or fixing bugs. Both subcategories provide significant efficiency gains but require careful validation, as generated outputs may introduce incorrect assumptions or incomplete logic. Studies indicate that developers often rely on these outputs for rapid progress, sometimes at the cost of reduced engagement with underlying problem details [1, 14].

3.4 Retrieval and Explanation Assistance

Retrieval and explanation assistance encompasses interaction modes where generative AI is used to access, summarize, or explain information relevant to programming tasks. Rather than generating new code directly, these interactions focus on supporting comprehension and decision-making by providing contextual knowledge, such as documentation summaries, error explanations, or conceptual clarifications [12]. This mode aligns closely with traditional information retrieval tasks but benefits from the contextual reasoning capabilities of LLMs [10].

This category can be divided into information retrieval and code explanation.

1. **Information retrieval** involves locating and summarizing relevant documentation or examples, reducing the need for manual search.
2. **Code explanation** focuses on interpreting existing code, errors, or outputs to help developers understand behavior and identify issues.

These subcategories are particularly valuable during debugging and learning phases, as they reduce cognitive overhead associated with searching and interpreting external resources. However, reliance on AI-generated explanations may affect how developers internalize knowledge and reason about code correctness [6].

References

- [1] S. Barke, M. B. James, and N. Polikarpova. Grounded copilot: How programmers interact with code-generating models, 2022. URL <https://arxiv.org/abs/2206.15000>.
- [2] C. Bird, D. Ford, T. Zimmermann, N. Forsgren, E. Kalliamvakou, T. Lowdermilk, and I. Gazit. Taking flight with copilot. *Commun. ACM*, 66(6):56–62, May 2023. ISSN 0001-0782. doi: 10.1145/3589996. URL <https://doi.org/10.1145/3589996>.
- [3] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, L. Gillespie, K. Goel, N. Goodman, S. Grossman, N. Guha, T. Hashimoto, P. Henderson, J. Hewitt, D. E. Ho, J. Hong, K. Hsu, J. Huang, T. Icard, S. Jain, D. Jurafsky, P. Kalluri, S. Karamcheti, G. Keeling, F. Khani, O. Khattab, P. W. Koh, M. Krass, R. Krishna, R. Kudithipudi, A. Kumar, F. Ladhak, M. Lee, T. Lee, J. Leskovec, I. Levent, X. L. Li, X. Li, T. Ma, A. Malik, C. D. Manning, S. Mirchandani, E. Mitchell, Z. Munyikwa, S. Nair, A. Narayan, D. Narayanan, B. Newman, A. Nie, J. C. Nibbles, H. Nilforoshan, J. Nyarko, G. Ogut, L. Orr, I. Papadimitriou, J. S. Park, C. Piech, E. Portelance, C. Potts, A. Raghunathan, R. Reich, H. Ren, F. Rong, Y. Roohani, C. Ruiz, J. Ryan, C. Ré, D. Sadigh, S. Sagawa, K. Santhanam, A. Shih, K. Srinivasan, A. Tamkin, R. Taori, A. W. Thomas, F. Tramèr, R. E. Wang, W. Wang, B. Wu, J. Wu, Y. Wu, S. M. Xie, M. Yasunaga, J. You, M. Zaharia, M. Zhang, T. Zhang, X. Zhang, Y. Zhang, L. Zheng, K. Zhou, and P. Liang. On the opportunities and risks of foundation models, 2022. URL <https://arxiv.org/abs/2108.07258>.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>.
- [5] K. Ferdowsifard, S. Barke, H. Peleg, S. Lerner, and N. Polikarpova. Loopy: interactive program synthesis with control structures. *Proc. ACM Program. Lang.*, 5(OOPSLA), Oct. 2021. doi: 10.1145/3485530. URL <https://doi.org/10.1145/3485530>.
- [6] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. Studying the effect of ai code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394215. doi: 10.1145/3544548.3580919. URL <https://doi.org/10.1145/3544548.3580919>.
- [7] T. Mao, D. Zhao, H. Tang, X. Wang, and H. Zhang. A large-scale empirical study of ai-generated code in real-world repositories, 2026. URL <https://arxiv.org/abs/2603.27130>.
- [8] McKinsey & Company. The economic potential of generative ai, 2023. URL <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-revolution>.
- [9] A. Miltner, S. Gulwani, V. Le, A. Leung, A. Radhakrishna, G. Soares, A. Tiwari, and A. Udupa. On the fly synthesis of edit suggestions. *Proc. ACM Program. Lang.*, 3(OOPSLA), Oct. 2019. doi: 10.1145/3360569. URL <https://doi.org/10.1145/3360569>.
- [10] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, M. Saari, Z. Zhang, H. Le, T. Quan, and P. Abrahamsson. Generative artificial intelligence for software engineering—a research agenda. *Software: Practice and Experience*, 55(11):1806–1843, 2025. doi: <https://doi.org/10.1002/spe.70005>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.70005>.
- [11] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirel. The impact of ai on developer productivity: Evidence from github copilot, 2023. URL <https://arxiv.org/abs/2302.06590>.
- [12] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson. Ai-powered code review with llms: Early results, 2025. URL <https://arxiv.org/abs/2404.18496>.

- [13] V. Raychev, M. Vechev, and E. Yahav. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14*, page 419–428, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327848. doi: 10.1145/2594291.2594321. URL <https://doi.org/10.1145/2594291.2594321>.
- [14] P. Vaithilingam, T. Zhang, and E. L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA '22*, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391566. doi: 10.1145/3491101.3519665. URL <https://doi.org/10.1145/3491101.3519665>.
- [15] E. Wastl. Advent of code, 2023. URL <https://adventofcode.com/>.